

REPO Simples — Implementação do microserviço Users (continuação)

Contexto: os itens **0) Pré-requisitos/estado** e **1) Governança baseline** já foram concluídos. A partir daqui, começamos a implementação **técnica** no **root do repositório** (repo simples, sem monorepo). Cada etapa traz **fundamentação** do porquê.

1) “Hello, service!” — scaffold do NestJS no root

Por quê: iniciar com um esqueleto enxuto que já publica **Swagger**, aplica **validação** na borda e expõe **healthcheck**, reduz retrabalho e melhora DX.

1.1 Criar o projeto

```
# na raiz do repo (vazio ou só com a governança)
npm i -g @nestjs/cli
nest new . --package-manager npm --strict # usa o diretório atual
# se o CLI recusar usar ".", crie em tmp e mova o conteúdo para o root
```

1.2 Ativar validação e Swagger (`src/main.ts`)

```
import { ValidationPipe } from '@nestjs/common';
import { SwaggerModule, DocumentBuilder } from '@nestjs/swagger';

// ... dentro do bootstrap()
app.setGlobalPrefix('api');
app.useGlobalPipes(new ValidationPipe({
  whitelist: true,
  forbidNonWhitelisted: true,
  transform: true,
})));

const config = new DocumentBuilder()
  .setTitle('Aurora – Users')
  .setVersion('1.0')
  .build();
const document = SwaggerModule.createDocument(app, config);
SwaggerModule.setup('api/docs', app, document);
```

Fundamento: validação precoce = menos erros em runtime; Swagger = contrato visível para QA/Dev/Doc.

1.3 Healthcheck

Crie `src/health/health.controller.ts` e registre um `HealthModule` no `AppModule`.

```
import { Controller, Get } from '@nestjs/common';
@Controller('health')
export class HealthController {
  @Get() liveness() { return { status: 'ok' }; }
}
```

1.4 Scripts no `package.json`

Garanta (além dos scripts padrão Nest):

```
{
  "scripts": {
    "start": "nest start",
    "start:dev": "nest start --watch",
    "build": "nest build",
    "lint": "eslint .",
    "test": "jest",
    "test:e2e": "jest --config ./test/jest-e2e.json",
    "migration:run": "typeorm-ts-node-commonjs -d src/infra/typeorm/data-source.ts migration:run",
    "migration:revert": "typeorm-ts-node-commonjs -d src/infra/typeorm/data-source.ts migration:revert"
  }
}
```

Abra um PR: `feat: nest skeleton (api/docs, validation, health)` → CI deve rodar e ficar verde.

2) Banco de dados & TypeORM (PostgreSQL) com Docker

Por quê: reprodutibilidade local/CI e isolamento por serviço.

2.1 Dependências

```
npm i @nestjs/typeorm typeorm pg
npm i -D ts-node @types/node
```

2.2 `docker-compose.yml` (raiz)

```
version: '3.8'
services:
```

```

db:
  image: postgres:15
  restart: unless-stopped
  environment:
    POSTGRES_USER: aurora
    POSTGRES_PASSWORD: aurora
    POSTGRES_DB: usersdb
  ports: [ "5432:5432" ]
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U aurora -d usersdb"]
    interval: 5s
    timeout: 3s
    retries: 10

```

2.3 `.env` e `.env.example`

```

# .env.example
DB_HOST=localhost
DB_PORT=5432
DB_USER=aurora
DB_PASS=aurora
DB_NAME=usersdb

```

Fundamento: padroniza variáveis e facilita onboarding de alunos.

2.4 DataSource (`src/infra/typeorm/data-source.ts`)

```

import 'dotenv/config';
import { DataSource } from 'typeorm';
export default new DataSource({
  type: 'postgres',
  host: process.env.DB_HOST,
  port: Number(process.env.DB_PORT ?? 5432),
  username: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
  entities: [__dirname + '/../**/*.entity.{ts,js}'],
  migrations: [__dirname + '/../migrations/*.ts,js'],
  synchronize: false,
  logging: false,
});

```

2.5 Conexão no AppModule

```

import { TypeOrmModule } from '@nestjs/typeorm';
import dataSource from '../infra/typeorm/data-source';

```

```
TypeOrmModule.forRoot({
  ...dataSource.options,
  autoLoadEntities: true,
});
```

3) Domínio User + CRUD

Por quê: entregar o MVP funcional do serviço “Users” com regras mínimas de segurança.

3.1 Entidade (src/users/user.entity.ts)

```
import { Entity, PrimaryGeneratedColumn, Column, CreateDateColumn,
UpdateDateColumn, VersionColumn, Index } from 'typeorm';

@Entity('users')
export class User {
  @PrimaryGeneratedColumn('uuid') id: string;

  @Index({ unique: true }) @Column({ length: 180 }) email: string;
  @Column({ length: 120 }) name: string;

  @Column({ select: false }) passwordHash: string; // nunca retornar

  @CreateDateColumn() createdAt: Date;
  @UpdateDateColumn() updatedAt: Date;
  @VersionColumn() version: number; // lock otimista
}
```

Fundamento: `unique` em email garante integridade; `select: false` evita vaziar hash.

3.2 DTOs (create-user.dto.ts / update-user.dto.ts)

```
import { IsEmail, IsNotEmpty, MinLength, IsOptional } from 'class-validator';
export class CreateUserDto {
  @IsEmail() email: string;
  @IsNotEmpty() name: string;
  @MinLength(8) password: string;
}
export class UpdateUserDto {
  @IsOptional() @IsEmail() email?: string;
  @IsOptional() @IsNotEmpty() name?: string;
  @IsOptional() @MinLength(8) password?: string;
}
```

3.3 Service (hash + conflitos)

```
import { Injectable, ConflictException, NotFoundException } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { User } from '../user.entity';
import { CreateUserDto, UpdateUserDto } from '../dto';
import * as bcrypt from 'bcrypt';

@Injectable()
export class UsersService {
  constructor(@InjectRepository(User) private repo: Repository<User>) {}

  async create(dto: CreateUserDto) {
    const exists = await this.repo.findOne({ where: { email: dto.email } });
    if (exists) throw new ConflictException('email already in use');
    const passwordHash = await bcrypt.hash(dto.password, 10);
    const user = this.repo.create({ email: dto.email, name: dto.name, passwordHash });
    const saved = await this.repo.save(user);
    delete (saved as any).passwordHash;
    return saved;
  }

  findAll() { return this.repo.find(); }

  async findOne(id: string) {
    const user = await this.repo.findOne({ where: { id } });
    if (!user) throw new NotFoundException();
    return user;
  }

  async update(id: string, dto: UpdateUserDto) {
    const user = await this.repo.findOne({ where: { id } });
    if (!user) throw new NotFoundException();
    if (dto.email && dto.email !== user.email) {
      const exists = await this.repo.findOne({ where: { email: dto.email } });
      if (exists) throw new ConflictException('email already in use');
      user.email = dto.email;
    }
    if (dto.name) user.name = dto.name;
    if (dto.password) user.passwordHash = await bcrypt.hash(dto.password, 10);
    const saved = await this.repo.save(user);
    delete (saved as any).passwordHash;
    return saved;
  }

  async remove(id: string) {
```

```

    const res = await this.repo.delete(id);
    if (!res.affected) throw new NotFoundException();
    return { id };
  }
}

```

3.4 Controller (rotas)

```

import { Controller, Get, Post, Put, Delete, Param, Body } from '@nestjs/common';
import { UsersService } from '../users.service';
import { CreateUserDto, UpdateUserDto } from '../dto';

@Controller('users')
export class UsersController {
  constructor(private service: UsersService) {}
  @Post() create(@Body() dto: CreateUserDto) { return
this.service.create(dto); }
  @Get() findAll() { return this.service.findAll(); }
  @Get('/:id') findOne(@Param('id') id: string) { return
this.service.findOne(id); }
  @Put('/:id') update(@Param('id') id: string, @Body() dto: UpdateUserDto) {
return this.service.update(id, dto); }
  @Delete('/:id') remove(@Param('id') id: string) { return
this.service.remove(id); }
}

```

Fundamento: controladores finos; regra de negócio no service.

4) Migrations (sem `synchronize:true`)

Por quê: previsibilidade entre ambientes e auditoria de mudanças no schema.

```

# subir o DB local
docker compose up -d db

# gerar migration inicial (tabelas/índices)
npx typeorm-ts-node-commonjs -d src/infra/typeorm/data-source.ts
migration:generate src/migrations/InitUsers

# aplicar
npm run migration:run

```

5) CI (GitHub Actions) com Postgres service

Por quê: reproduzir o setup local no pipeline e garantir os 3 **checks** exigidos.

`.github/workflows/ci.yml` (essência):

```
name: CI
on:
  pull_request: { branches: [ main ] }
  push: { branches: [ main ] }

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm run lint

  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm run build

  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_USER: aurora
          POSTGRES_PASSWORD: aurora
          POSTGRES_DB: usersdb
        ports: [ "5432:5432" ]
        options: >-
          --health-cmd="pg_isready -U aurora -d usersdb"
          --health-interval=5s --health-timeout=3s --health-retries=10
    env:
      DB_HOST: localhost
      DB_PORT: 5432
      DB_USER: aurora
      DB_PASS: aurora
      DB_NAME: usersdb
    steps:
```

```

- uses: actions/checkout@v4
- uses: actions/setup-node@v4
  with: { node-version: '20', cache: 'npm' }
- run: npm ci
- name: Run migrations
  run: npm run migration:run
- name: Test
  run: npm test

```

Fundamento: três jobs = contexts claros (`lint`, `build`, `test`) para a Branch Protection; banco real no job de testes evita falso-positivo.

6) Documentação (README) mínima

Por quê: onboarding rápido e padronização para a disciplina.

Inclua: - **Introdução** e escopo (Users Service). - **Como rodar local:** `docker compose up -d db` → `npm run start:dev`. - **Variáveis de ambiente** e `.env.example`. - **Migrations:** gerar/aplicar; decisões (sem `synchronize`). - **Endpoints:** `POST/GET/GET:id/PUT:id/DELETE:id`, `GET /health`, docs em `/api/docs`. - **Checklist de Auditoria** (já adicionado antes). - **Evidências:** links do CI, prints da proteção da `main`.

7) Fluxo de branches, commits e PRs (modo solo)

Por quê: histórico limpo e rastreável.

Branches sugeridas e ordem: 1. `feat/nest-skeleton` 2. `feat/db-typeorm` 3. `feat/users-crud` 4. `chore/ci` 5. `docs/readme-auditoria`

Regras práticas: - Commits **pequenos e descritivos**. - Sempre abrir **PR** (mesmo solo) → CI verde → **Squash and merge**. - Branch é apagada automaticamente após merge.

8) Definição de Pronto (DoD)

- `/api/docs` ativo; `ValidationPipe` global; `GET /health` ok.
- CRUD Users funcional; email único; senha com **bcrypt**; **hash não exposto**.
- Migrations geradas e aplicadas (local e CI).
- CI verde nos 3 jobs; proteção da `main` exigindo PR + checks + `strict` + linear; sem force push/deletion.
- README atualizado com evidências e instruções de execução.

9) Próximos passos

- **E2E tests** mínimos (`supertest`) para `POST /users` e `GET /users/:id`.

- **Paginação** em `GET /users` e índices adequados.
- **Observabilidade** básica (logger estruturado, requestId).
- Preparar o terreno para **outros microserviços** (mesma governança; repos separados ou migração para monorepo no futuro).